

RESEARCH ARTICLE

A Multi-Level Acceleration Scheme for AI Model Training on ARM Architecture Processors

Yang Bai¹  | MingDong Xie¹  | Meng Hao¹ | WeiZhe Zhang¹  | HongWei Yang¹ | Li He² | Wei Duan² | NingCheng Wang²

¹School of Cyberspace Science, Harbin Institute of Technology, Harbin, China | ²State Key Laboratory of Mobile Network and Mobile Multimedia Technology, ZTE, Shenzhen, China

Correspondence: Meng Hao (haomeng@hit.edu.cn)

Received: 18 November 2025 | **Revised:** 23 January 2026 | **Accepted:** 9 February 2026

Keywords: AI training acceleration | ARM architecture | convolutional neural network | im2col | mixed-precision training | mpack | SIMD

ABSTRACT

With the widespread application of reinforcement learning and deep learning on edge devices, training neural networks on ARM architecture processors has become an urgent demand. However, existing mainstream deep learning frameworks are not sufficiently optimized for training workloads on ARM CPUs, resulting in low training efficiency. To address this problem, this paper proposes and implements a multi-level AI training acceleration scheme based on the open-source C++ library mpack, targeting the slow training speed and high resource consumption of Convolutional Neural Networks (CNNs) on ARM platforms. The scheme accelerates training by deeply optimizing the im2col algorithm to convert convolutions into efficient matrix multiplications, utilizing ARM NEON SIMD instructions to optimize linear operators, and integrating an FP64/FP16 mixed-precision training strategy with dynamic loss scaling. Experimental results on LeNet-5 and VGG11-style CNNs show substantial performance gains over the original mpack and mainstream frameworks. On an NVIDIA Jetson AGX Orin, our implementation achieves up to 7.3× speedup over the original mpack baseline and up to 11.3× and 5.69× end-to-end speedups over PyTorch and TensorFlow, respectively, while still delivering multi-fold reductions in training time on a low-resource Raspberry Pi platform. In DQN-based reinforcement learning for Atari Breakout, our solution attains a 4.98× end-to-end speedup over the PyTorch single-threaded baseline and maintains 2.37× and 4.23× advantages over 4-threaded PyTorch and TensorFlow implementations. Ablation studies confirm the complementary nature of the proposed optimizations, with convolutional, linear, and mixed-precision components jointly contributing to the overall speedup and enabling an attractive performance–accuracy trade-off for ARM-based edge computing.

1 | Introduction

Artificial intelligence (AI) technology has permeated numerous fields such as healthcare, military, education, and finance, profoundly impacting global development. Among the many deep learning methods, Convolutional Neural Networks (CNNs) are the most widely used. By stacking convolutional layers, CNN models can effectively learn high-level abstract features from large datasets to intelligently solve complex problems [1, 2].

The performance of these layers, particularly convolutional and fully-connected layers, directly dictates the speed of network training and inference.

With the proliferation of 5G technology and continuous improvements [3] in CPU performance, deploying deep neural networks on network elements, mobile devices, and embedded systems has become a feasible paradigm known as Edge Intelligence [4], which offers users convenient and personalized services.

While this trend offers users convenient and personalized services [5], it also introduces significant challenges in both computational efficiency and communication overhead across wireless networks [6]. The application scenarios for CNNs have expanded to cover industries such as medical diagnostics, autonomous driving, and voice assistants. For instance, the widespread adoption of smart Internet of Things (IoT) devices has benefited from CNN optimization techniques like network pruning [7], quantization [8–10], and efficient architecture design [11–14], enabling deep learning on low-power CPUs for edge computing. Similarly, on mobile devices, lightweight CNNs provide faster inference and longer battery life for tasks like image recognition and real-time translation. The advancement of autonomous driving is also inseparable from the lightweight development of CNNs [15].

However, most of these edge and mobile devices rely on ARM architecture processors, which typically lack dedicated acceleration hardware and thus face challenges of limited computational power. While a significant body of research has focused on inference acceleration [16–18], the acceleration of the model training process on these platforms has received comparatively little attention. This research gap is particularly critical as efficient on-device training serves as a foundational component for emerging distributed learning paradigms like Federated Learning [19]. This gap is further exacerbated by the limitations of existing frameworks. For instance, PyTorch, while dominant, introduces significant performance overhead due to its Python-centric design and compatibility-focused abstractions, making it sub-optimal for resource-constrained ARM environments [20]. Conversely, native C++ libraries like mlpack offer higher raw performance, but their default implementations fail to leverage ARM-specific hardware features like NEON instructions, leaving substantial performance potential untapped [21].

To address this research gap, the primary objective of this paper is to design, implement, and validate a multi-level AI training acceleration scheme for ARM platforms. By integrating deep optimizations at the algorithmic, instruction-set, and strategic levels, this scheme aims to unlock the full performance potential of native C++ libraries on resource-constrained devices and achieve training speeds that surpass mainstream, general-purpose frameworks. To this end, we select mlpack as our foundational platform due to its high performance, C++-native architecture, and customizability, which provide an ideal environment for the fine-grained optimizations central to our work.

To achieve this objective, this paper makes several key contributions. First, we perform a deep optimization of core computational layers for ARM CPUs by re-engineering the im2col algorithm for efficient matrix multiplication and implementing vectorized linear operators with ARM NEON intrinsics, significantly enhancing parallelism and hardware utilization. Second, we design and integrate a novel mixed-precision training strategy tailored to mlpack, which combines FP64 master weights with lower-precision computations to maximize throughput while ensuring numerical stability [22]. Finally, we implement and comprehensively validate these optimizations within an end-to-end accelerated framework, demonstrating through extensive experiments that our solution decisively outperforms both original mlpack and mainstream PyTorch in CNN and reinforcement learning training scenarios on ARM

platforms. Ultimately, this work provides an effective solution for deploying advanced deep learning training tasks on edge and mobile devices, enhancing the capabilities of ARM-based systems in the AI landscape.

The rest of this paper is organized as follows: Section 2 reviews related work. An overview of our proposed design is provided in Section 3. Following this, the optimizations for linear and convolutional operators are detailed in Sections 4 and 5, respectively. The implementation of our mixed-precision training strategy is described in Section 6. The experimental results and a corresponding analysis are presented in Section VII. Finally, Section VIII concludes the paper and discusses future work.

2 | Related Work

To achieve efficient AI model training on resource-constrained ARM platforms, prior research has primarily advanced along two dimensions: The software level, through the development and optimization of deep learning frameworks, and the hardware level, by leveraging processor-specific instruction sets to accelerate underlying computations. This section reviews the state-of-the-art in both areas to contextualize the contributions of this paper.

2.1 | Overview of Mainstream Deep Learning Frameworks

The deep learning framework landscape is currently dominated by industry giants like PyTorch and TensorFlow, each with distinct strengths. PyTorch is widely favored in academia for its Python-native feel and flexible dynamic computation graph, making it ideal for research and prototyping [20]. TensorFlow, a powerhouse in production environments, offers TensorFlow Lite (TFLite), a specialized solution for ARM devices [23]. However, for our research focus on deep, on-device training optimization, both present limitations. PyTorch's Python-centric nature can introduce overhead, and while its C++ frontend exists, it is less direct for deep, low-level hardware tuning. Meanwhile, TFLite is highly optimized for model inference, but its on-device training capabilities are less developed.

In contrast, mlpack, a fast and flexible machine learning library written entirely in C++, offers a distinct advantage for our work [21]. Built on the high-performance Armadillo linear algebra library [24], it avoids the overhead of interpreted languages and provides a direct path for low-level hardware optimization. This C++-native structure makes it an ideal platform for the deep algorithmic and instruction-set-level modifications central to our work, which are more challenging to implement within the ecosystems of PyTorch or TensorFlow. A concise summary and direct comparison of their key features is provided (Table 1).

2.2 | Hardware Acceleration on ARM Architecture

The ARM architecture, dominant in mobile and embedded computing, has evolved to include powerful features specifically for

TABLE 1 | Comparison of mainstream deep learning frameworks, including programming languages, computation graphs, and deployment capabilities across different platforms.

Feature	PyTorch	TensorFlow	mlpack
Primary language	Python	Python, C++	C++
Computation graph	Dynamic	Static/Dynamic	Algorithm-oriented
Core strengths	Flexible, research-friendly, active community	Mature ecosystem, powerful for production deployment	High-performance, lightweight, C++ native, easy to integrate
ARM support	Primarily via LibTorch or mobile deployment solutions	TensorFlow Lite is optimized for inference	Native support, easy for cross-compilation
Primary use case	Rapid prototyping, academic research	Industrial-scale deployment, mobile inference	High-Performance Computing (HPC), embedded systems, algorithm research
Customizability	High	High, but framework structure is complex	Extremely High (allows direct modification of C++ source code and algorithms)

accelerating machine learning workloads. The most fundamental of these is the NEON technology, a Single Instruction, Multiple Data (SIMD) architecture. NEON engines, typically featuring 128-bit registers, can perform the same operation on multiple data elements simultaneously. This is exceptionally effective for the matrix multiplications and convolutions that form the computational core of deep learning, offering significant speedups over scalar operations [25]. Building upon this, ARM introduced the Scalable Vector Extension (SVE) and its successor, SVE2, which offer vector-length agnostic programming to ensure future-proofing and efficiency across a wider range of hardware.

However, unlocking the full potential of these SIMD instructions requires more than just their direct application; it critically depends on how data is organized in memory. The performance of deep learning models, especially CNNs, is highly sensitive to data layout as it directly impacts memory access patterns, cache utilization, and the vectorization efficiency of SIMD units [26]. The most common layouts for 4D tensor data (Batch, Channel, Height, Width) are NCHW and NHWC. While NHWC can be advantageous for convolutions by allowing all channel values for a single spatial location to be read contiguously, neither layout is universally optimal.

To enhance cache locality and parallel computation, researchers and framework developers often employ more specialized or transformed data layouts. Formats like NCHWc8 or NCHWc16 pack a small number of channels together, creating channel blocks that can be processed efficiently in parallel by SIMD units. A significant challenge arises from the fact that different network layers may have different layout preferences. This necessitates data layout transformations between layers, which introduce overhead that can offset the performance gains from SIMD optimizations. Therefore, minimizing the frequency and cost of these transformations, or designing a unified layout that is friendly to multiple operations, is a key consideration in ARM platform optimization. Works such as LIBSHALOM, which optimizes small and irregular matrix multiplications on ARMv8 multi-cores [27], and recent efforts to optimize attention mechanisms by exploiting data reuse on ARM CPUs [28], directly or indirectly

involve efficient data organization strategies tailored to ARM hardware.

Furthermore, as ARM processors expand into high-performance computing, multi-core and multi-socket systems increasingly adopt Non-Uniform Memory Access (NUMA) architectures. This introduces another layer of optimization complexity. Research has focused on NUMA-aware strategies, such as load balancing across nodes and optimizing memory access patterns to improve performance on multi-core processors [29]. In practice, mobile-focused deep learning frameworks like TensorFlow Lite and libraries such as the Arm Compute Library (ACL) are continuously refining their data repacking strategies to ensure efficient execution across diverse hardware conditions [30].

In addition to general-purpose deep learning frameworks and vendor libraries, there also exist specialized research efforts that target convolution acceleration on ARM CPUs. For example, Zhou et al. [31] propose a pipelining strategy that overlaps data loading and computation to speed up CNNs on ARM processors. Their work primarily optimizes standalone convolution kernels for inference and forward computation, whereas our work focuses on an end-to-end training framework built on mlpack, redesigning the operator stack via an im2col+GEMM formulation and integrating mixed-precision training.

On the library side, Arm Ltd. provides the ARM Compute Library (ACL), a collection of highly optimized math kernels (e.g., convolutions and NEGEMM) that is widely regarded as a performance upper bound for hand-written code on ARM CPUs and is heavily used in mobile inference stacks such as PyTorch Mobile and TensorFlow Lite. In this paper, we treat ACL as a vendor-optimized baseline at the GEMM level and explicitly compare its NEGEMM implementation with our custom FP16 GEMM kernels that underpin the im2col-based convolutions in our multi-level acceleration scheme.

In summary, the ARM architecture provides a rich set of hardware features for ML acceleration, and both research prototypes and vendor libraries (e.g., ACL) have explored how to exploit

them at the operator level. Nevertheless, a noticeable gap remains between this hardware potential and its realization in end-to-end training systems: Mainstream frameworks often prioritize portability over tight hardware co-design, and existing ARM-oriented works typically focus on standalone convolution kernels or inference-only pipelines. This gap motivates our work on a holistic, training-oriented acceleration scheme that co-optimizes data layout, im2col+GEMM operators, and mixed-precision strategies to better align advanced ARM hardware with the practical demands of on-device AI training.

3 | Design Overview

This research is dedicated to exploring and implementing efficient training acceleration for Convolutional Neural Network (CNN) models on ARM architecture processors. The study is based on mlpack, a mature and widely-used open-source C/C++ machine learning library. It involves an in-depth analysis and optimization of the library's core training pipeline to fully leverage the computational characteristics of ARM processors and to overcome the bottlenecks associated with training complex models on edge devices. The research is primarily composed of three technical modules: Deep Optimization of Convolutional Operators, Efficient Implementation of Linear Operators, and Mixed-Precision Training.

As illustrated in the system architecture (Figure 1), the main research consists of several key components. The first is a deep optimization of convolutional operators, which targets the most computationally intensive part of a CNN. By improving the im2col algorithm and integrating the data processing

pipeline, we transform multiple, dispersed convolutions into a single, large-scale matrix multiplication and optimize its parallel execution efficiency. For fully connected layers and other parts involving linear operations, core matrix operations are then rewritten using vectorization with the ARM NEON SIMD instruction set, with a particular focus on accelerating half-precision (FP16) floating-point calculations.

Building upon this foundation, we also introduce the FP16 data type into mlpack's existing double-precision framework to accelerate compute-intensive operations. This method is combined with loss scaling techniques to further improve training speed while preserving the stability of the master weights, which are maintained in double precision.

4 | Efficient Implementation of Linear Operators

Linear operators, also known as Basic Linear Algebra Subprograms (BLAS) computations, are a primary source of computational load in Convolutional Neural Networks (CNNs). Their application is not limited to fully connected layers; they are also utilized in specially optimized convolutional layers, for instance, through the im2col algorithm. Optimizing these linear operators is therefore crucial for enhancing the overall performance of CNNs.

A core optimization in this research involves rewriting these linear operators to bypass the performance bottlenecks of general-purpose libraries like OpenBLAS under specific conditions. By directly leveraging the vectorization (SIMD) capabilities of the

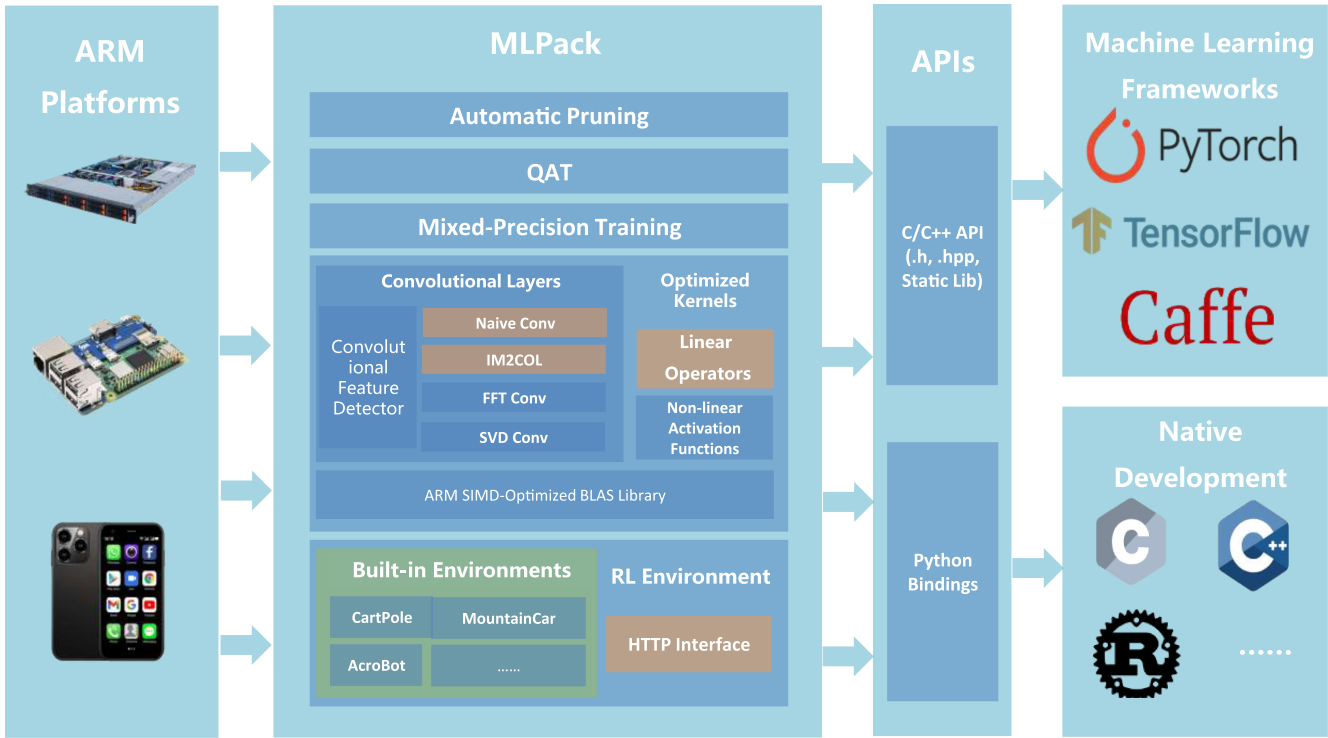


FIGURE 1 | The overall architecture of our proposed multi-level acceleration scheme. It integrates optimizations at the operator, precision, and model structure levels.

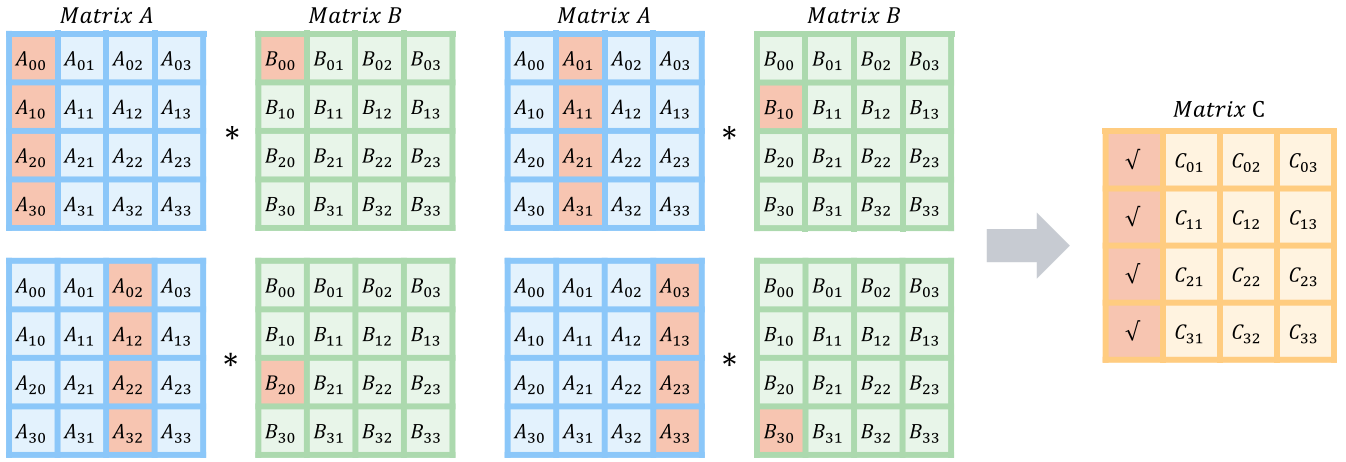


FIGURE 2 | Computation process of the VFMAQ scheme using FP16 precision and NEON vectorization. The diagram shows how matrix operations are parallelized through NEON's 128-bit registers to accelerate GEMM computations.

target ARM hardware, specifically the NEON instruction set, and employing low-precision data formats, we create customized, high-efficiency operators. NEON's 128-bit vector registers enable parallel processing of multiple data elements, forming the foundation for high-performance computing on mobile devices. Instead of relying on the underlying libraries, this study presents two such custom General Matrix Multiplication (GEMM) implementations using NEON intrinsics: A VFMAQ scheme based on half-precision floating-point (FP16) and a VDOT scheme based on 8-bit integers (int8), both designed to substantially boost computational speed on ARM platforms without significantly compromising model accuracy.

4.1 | VFMAQ Scheme Based on Half-Precision Floating-Point (FP16)

This scheme aims to improve computational efficiency by optimizing data layout and memory access patterns, combined with the use of the half-precision floating-point (FP16) data type. The mpack library uses a column-major memory storage format. To maximize the benefit of this characteristic, our algorithm performs computations on a column-by-column basis, thereby ensuring contiguous memory access. The core of the algorithm is a vectorized loop that processes the matrix multiplication $C = A \times B$ by operating on vertical strips of data (Figure 2 and Algorithm 1).

The computational process begins by initializing the output matrix C to zeros (Algorithm 1 Line 1) and calculating the starting index for handling tail elements (Line 2). The outer loop iterates through each column of the output matrix in parallel (Line 3), leveraging OpenMP to utilize multi-core CPUs. For each element in the current column of matrix B (Line 4), the algorithm loads a scalar value (Line 5). The core computation occurs in the vectorized loop (Lines 6–10), which processes eight elements at a time: It loads vectors from matrices C and A (Lines 7 and 8), performs the fused multiply-accumulate operation using the NEON intrinsic (Line 9), and stores the result back (Line 10). For the remaining elements that don't fit a full vector (Lines 11–13), a scalar loop ensures computational correctness.

ALGORITHM 1 | NEON Vectorized Half-Precision GEMM (VFMAQ Scheme).

```

Require:  $A \in \mathbb{R}^{M \times K}, B \in \mathbb{R}^{K \times N} \triangleright (\_fp16)$ 
Ensure:  $C \in \mathbb{R}^{M \times N} \triangleright (\_fp16)$ 
1: Initialize C to zeros
2: remain_start  $\leftarrow M - (M \bmod 8)$ 
3: for parallel  $j = 0 \rightarrow N - 1$  do
4:   for  $k = 0 \rightarrow K - 1$  do
5:      $b \leftarrow B[k, j]$ 
6:     for  $i = 0 \rightarrow \text{remain\_start} - 1$  step 8 do
7:        $\text{vecC} \leftarrow \text{vld1q\_f16}(C[i : i + 7, j])$ 
8:        $\text{vecA} \leftarrow \text{vld1q\_f16}(A[i : i + 7, k])$ 
9:        $\text{vecC} \leftarrow \text{vfmaq\_n\_f16}(\text{vecC}, \text{vecA}, b)$ 
10:       $\text{vst1q\_f16}(C[i : i + 7, j], \text{vecC})$ 
11:    end for
12:    for  $i = \text{remain\_start} \rightarrow M - 1$  do
13:       $C[i, j] \leftarrow C[i, j] + A[i, k] \cdot b$ 
14:    end for
15:  end for
16: end for
17: return C

```

This column-wise processing strategy is the source of its efficiency, as it perfectly aligns with mpack's column-major memory layout. By accessing contiguous blocks of memory from matrices A and C, the algorithm maximizes data locality and significantly improves the cache hit rate. For rows that do not fit into a full vector of 8 (the "tail" data), a separate scalar loop ensures correctness. This scheme also exhibits good hardware compatibility, with low requirements for the ARM instruction set version (supported on ARMv8-A and later), making it widely applicable.

4.2 | VDOT Scheme Based on 8-Bit Integer (Int8)

To pursue ultimate performance, we further explored an optimization path using 8-bit integer (int8) quantization and specialized hardware instructions. The core idea of the VDOT scheme is to use the dot product (dot) instruction to compute one

output element at a time, while achieving a high degree of vectorization within that computation. The detail is in Algorithm 2. The core idea of the VDOT scheme is to efficiently compute each element $C[i, j] = \sum_k A[i, k] \cdot B[k, j]$ through a high degree of vectorization within that computation (Algorithm 2 and Figure 3).

The computational process begins by initializing the output matrix C to zero (Algorithm 2 Line 1). The outer loop parallelizes over rows of matrix A (Line 2), while the inner loop iterates through columns of matrix B (Line 3). For each output element $C[i, j]$, the algorithm initializes a vector accumulator (Line 4) and calculates the starting index for handling tail elements along the K dimension (Line 5). The core computation occurs in the vectorized loop (Lines 6–10), which processes 16 elements at a time: It loads 16-element int8 vectors from row i of matrix A and row j of the transposed matrix B_t (Lines 7 and 8), then performs four parallel 4-element dot products using the `vdotq_s32` intrinsic (Line 9). After the vectorized loop, horizontal addition sums the

ALGORITHM 2 | NEON Vectorized int8 GEMM using VDOT.

Require: $A \in \mathbb{Z}^{M \times K}$, $B_t \in \mathbb{Z}^{N \times K}$
Ensure: $C \in \mathbb{Z}^{M \times N}$

```

1:  $C \leftarrow 0$ 
2: for parallel  $i=0$  to  $M-1$  do
3:   for  $j=0$  to  $N-1$  do
4:      $\text{sum\_vec} \leftarrow \{0, 0, 0, 0\}$ 
5:      $\text{remain\_k\_start} \leftarrow K - (K \bmod 16)$ 
6:     for  $k=0$  to  $\text{remain\_k\_start}-1$  step 16 do
7:        $\text{vecA} \leftarrow A[i, k : k+15]$ 
8:        $\text{vecB\_t} \leftarrow B_t[j, k : k+15]$ 
9:        $\text{sum\_vec} \leftarrow \text{vdotq\_s32}(\text{sum\_vec}, \text{vecA}, \text{vecB\_t})$ 
10:    end for
11:     $\text{final\_sum} \leftarrow \text{sum\_vec}[0] + \text{sum\_vec}[1] + \text{sum\_vec}[2] + \text{sum\_vec}[3]$ 
12:    for  $k=\text{remain\_k\_start}$  to  $K-1$  do
13:       $\text{final\_sum} \leftarrow \text{final\_sum} + A[i, k] \cdot B_t[j, k]$ 
14:    end for
15:     $C[i, j] \leftarrow \text{final\_sum}$ 
16:  end for
17: end for
18: return  $C$ 

```

four partial results in the accumulator (Line 11), while a scalar loop handles any remaining tail data (Lines 12–14). The final result is stored in the output matrix (Line 15).

This scheme employs several key techniques to achieve high performance. The input matrices use int8 data type with int32 accumulation to prevent overflow, and matrix B is transposed to enable efficient memory-contiguous access. The primary advantage lies in exceptional performance: int8 doubles data throughput compared to FP16, and the `vdot` instruction completes multiple multiply-accumulate operations per cycle. However, the scheme has a strong hardware dependency, as the core `vdot` instruction requires ARMv8.4-A architecture, limiting its universality on older embedded processors.

The VDOT-based GEMM kernel relies on the ARMv8.4-A dot product extension and is therefore only available on relatively recent cores such as Cortex-A75 and Cortex-A78AE. To ensure portability across a broader range of edge devices, including widely deployed Cortex-A53 and Cortex-A72 CPUs that lack native dot product instructions, our implementation incorporates an automatic fallback mechanism. At compile time or during runtime feature detection, if the VDOT instruction set is not present, the library transparently switches from the int8 VDOT kernel to the FP16 VFMAQ-based GEMM described in Section 4.1 (or to a scalar int8 accumulation path where appropriate). This guarantees functional correctness on all ARMv8-A platforms, while still allowing devices that support dot product instructions to exploit the more aggressive VDOT scheme for maximum performance.

5 | In-Depth Optimization of Convolutional Operators

As the core component of Convolutional Neural Networks (CNNs), the computational efficiency of the convolution operation directly determines the network's overall training and inference performance. Traditional naive convolution implementations are often inefficient due to their inherent nested loops and non-contiguous memory access patterns. These characteristics prevent the full utilization of the parallel computing potential found in modern processors, leading to significant performance degradation, especially on edge processors like ARM with smaller caches.

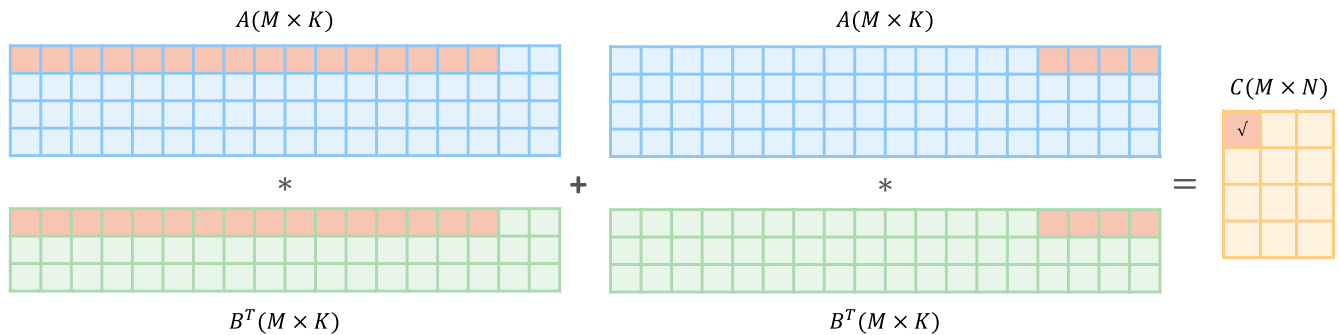


FIGURE 3 | Computation workflow of the VDOT scheme using int8 quantization. The diagram shows how 8-bit integer vectors are processed through NEON's dot product instructions to accelerate matrix multiplication on ARM architectures.

To overcome this bottleneck, we introduce the im2col algorithm, an algorithm that transforms the traditional convolution into a single matrix multiplication. This allows us to leverage the specially optimized linear operators based on the ARM NEON instruction set that we proposed in the previous section. At the same time, we also apply a special computational architecture to reduce data redundancy, optimize memory access, and improve parallel computing efficiency. Through these optimizations, our implementation significantly enhances the execution performance of convolutional operators within CNN models on ARM architecture processors.

5.1 | The Im2col Algorithm

A standard 2D convolution extracts features by sliding a kernel across an input feature map (Figure 4). However, a naive implementation of this operation typically involves multiple nested loops, which results in non-contiguous memory access patterns. This pattern severely degrades cache hit rates and instruction pipeline efficiency, thereby limiting performance.

To overcome this bottleneck, our work employs the im2col algorithm (Figure 5). The core idea of this algorithm is to transform the convolution into one or more efficient General Matrix Multiplications (GEMM) through data rearrangement. Specifically, each local region of the input feature map that interacts with the kernel is “unrolled” into a column vector. These column vectors are then concatenated to form a large matrix (the Col matrix). Concurrently, all convolution kernels are flattened into another matrix (the Filt matrix), where each row corresponds to a single kernel. The original convolution operation thus becomes equivalent to the multiplication of these two flattened matrices. This transformation allows the computation to directly leverage highly optimized low-level libraries, such as BLAS, or SIMD instruction sets, leading to a significant performance increase.

5.2 | Optimization of the Convolution Operator

In many conventional machine learning frameworks, the convolution operator is often implemented in a way that requires repeatedly reading the same memory locations (Algorithm 3). This approach typically enumerates each convolution position

and then dispatches the computation to a suitable kernel algorithm. While this provides significant operator flexibility, it requires repeatedly reading the same memory locations, a pattern that is particularly ill-suited for small neural networks on edge devices like ARM, which have limited cache sizes. After performance profiling the original forward pass in mlpack, we identified this fine-grained execution strategy as a critical bottleneck. In the context of im2col, this translates to generating a tiny intermediate matrix for each input-channel/output-kernel pair, leading to frequent data copy and memory allocation overhead, and failing to effectively leverage the large-scale parallel computing capabilities of modern processors. Therefore, our optimization strategy was designed to be self-contained within the convolution layer’s implementation file to minimize invasiveness and ensure compatibility.

To address this, we propose a **unified batched GEMM strategy**, refactoring the forward, backward, and gradient computations into large-scale matrix multiplications.

- **Forward Pass:** We convert all input channels of a single sample into one large Col matrix via im2col and organize all kernels into a Filt matrix, executing the entire forward computation efficiently with a single GEMM operation ($\text{Out} = \text{Filt} \times \text{Col}$) (Algorithm 4). The process begins by computing output dimensions and padding the input (Lines 1–3). For each batch sample (Line 4), it constructs a Col matrix (Lines 5–13), reshapes kernels into a 2D matrix (Line 14), performs convolution via a single GEMM (Line 15), and reshapes the result into the final output feature map (Lines 16–18).
- **Backward Pass:** Adopting a similar GEMM-based strategy, the backward pass computes the input gradients. This is achieved by multiplying the incoming output gradients (gradOut) with the transposed kernel matrix (Filt^T), followed by a col2im transformation to restore the gradients’ spatial structure.
- **Gradient Pass:** The gradient computation for the kernels follows a comparable logic. It is also transformed into a single GEMM operation, where the output gradients (gradOut) are multiplied with the transposed input Col matrix (Col^T) to calculate the updates for all kernels at once.

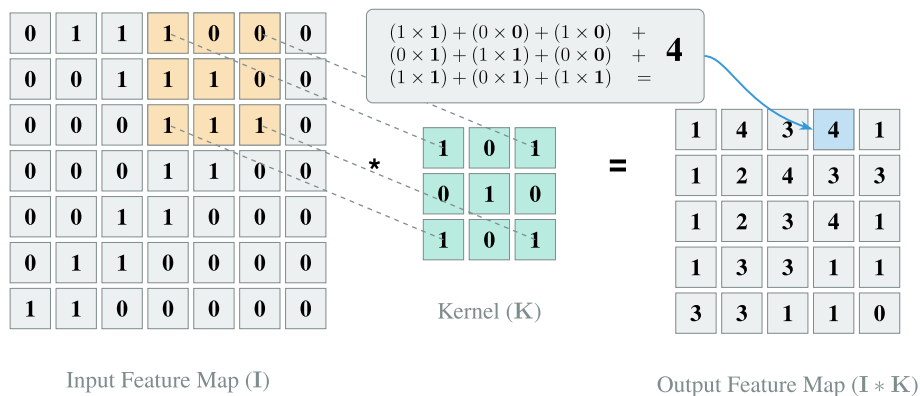


FIGURE 4 | An illustration of the 2D convolution operation. The kernel is applied to a patch of the input feature map to produce a single value in the output feature map.

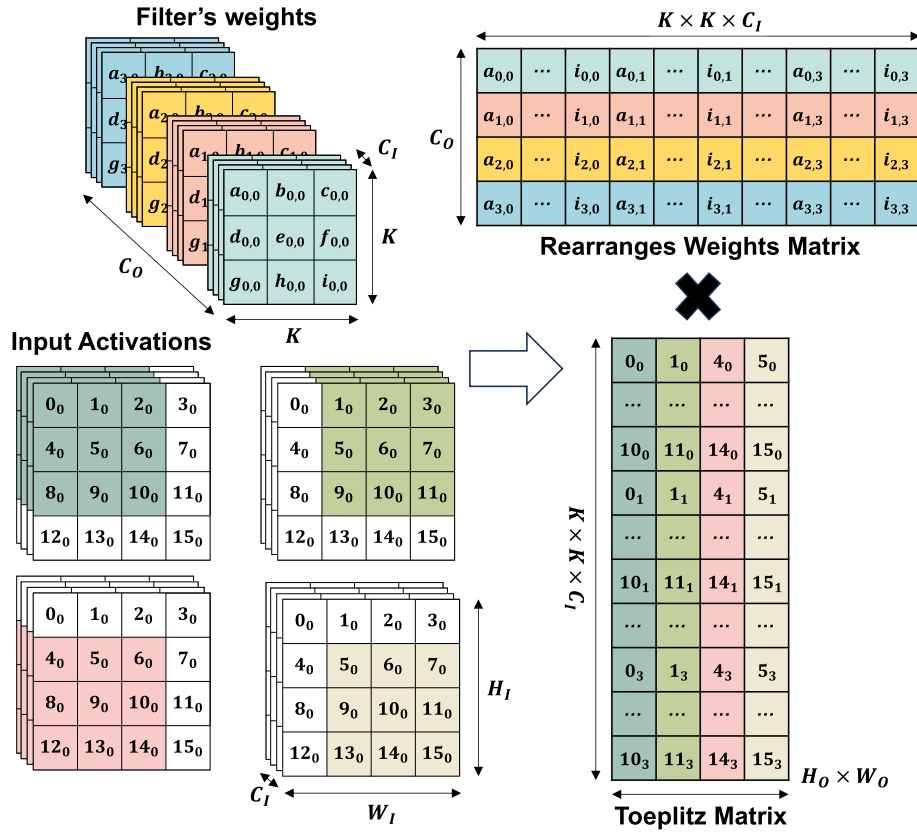


FIGURE 5 | Transformation process of the im2col algorithm that converts convolution operations into GEMM. This diagram illustrates how local input regions are unrolled into columns to enable efficient matrix multiplication.

ALGORITHM 3 | Original convolution algorithm in mlpack's forward propagation.

```

1: for offset from 0 to (higherInDimensions * batchSize - 1) do
2:   fullInputOffset ← offset * inMaps
3:   fullOutputOffset ← offset * maps
4:   for outMap from 0 to (maps - 1) do
5:     convOutput ← outputTemp.slice(outMap + fullOutputOffset)
6:     for inMap from 0 to (inMaps - 1) do
7:       Apply the convolution rule:
8:       Input: inputTemp.slice(inMap + fullInputOffset)
9:       Weight: weight.slice(outMap * inMaps + inMap)
10:      Output: convOutput
11:      Stride: (strideWidth, strideHeight)
12:      Padding: (1, 1)
13:     end for
14:   end for
15: end for

```

By unifying these three core processes into large-scale GEMM operations, we construct an end-to-end efficient computational pipeline that comprehensively addresses the performance bottlenecks of the original implementation.

6 | Mixed-Precision Training

Low-precision computation, particularly using half-precision floating-point (FP16), offers significant performance advantages

on ARM platforms by leveraging NEON SIMD instructions for higher throughput and reduced memory bandwidth. However, directly replacing standard single-precision (FP32) with FP16 throughout the training process can lead to severe numerical errors, such as gradient underflow or overflow, causing training instability.

To address this, mixed-precision training has become a mainstream solution. Unfortunately, the mlpack framework, which this study is based on, lacks any native support for such a scheme.

ALGORITHM 4 | Optimized Batched im2col-based Convolution.

Require: *input_maps, filters, biases, stride, padding*
Ensure: *output_feature_maps*

```

1:  $out_{height} \leftarrow (input_{height} + pad_{top} + pad_{bottom} - kernel_{height}) / stride_h + 1$ 
2:  $out_{width} \leftarrow (input_{width} + pad_{left} + pad_{right} - kernel_{width}) / stride_w + 1$ 
3:  $padded_{input} \leftarrow pad(input\_maps, pad_{top}, pad_{bottom}, pad_{left}, pad_{right})$ 
4: for  $batch_{index}$  from 0 to  $batch_{size} - 1$  do
5:   Initialize  $col_{matrix}$  with shape  $(kernel_{height} \cdot kernel_{width} \cdot in_{channels}, out_{height} \cdot out_{width})$ 
6:    $col_{index} \leftarrow 0$ 
7:   for  $y$  from 0 to  $out_{height} - 1$  do
8:     for  $x$  from 0 to  $out_{width} - 1$  do
9:       for  $c$  from 0 to  $in_{channels} - 1$  do
10:         $patch \leftarrow padded_{input}[batch_{index}, c, y \cdot stride_h : y \cdot stride_h + kernel_{height}, x \cdot stride_w : x \cdot stride_w + kernel_{width}]$ 
11:         $col_{matrix}[:, col_{index}] \leftarrow patch.flatten()$ 
12:      end for
13:       $col_{index} \leftarrow col_{index} + 1$ 
14:    end for
15:  end for
16:   $filter_{matrix} \leftarrow filters.reshape(out_{channels}, kernel_{height} \cdot kernel_{width} \cdot in_{channels})$ 
17:   $intermediate_{result} \leftarrow filter_{matrix} \times col_{matrix}$ 
18:   $output_{slice} \leftarrow intermediate_{result}.reshape(out_{channels}, out_{height}, out_{width})$ 
19:   $output\_feature\_maps[batch_{index}, :, :, :] \leftarrow output_{slice}$ 
20: end for
21: return output_feature_maps

```

Its core data structures rely on the Armadillo library, which defaults to double-precision (FP64) and does not support the FP16 type. Our initial attempts to force a global FP32 environment resulted in immediate numerical collapse (loss becoming NaN). To harness the performance benefits of FP16 on ARM, we therefore designed and implemented a novel mixed-precision training framework specifically adapted for mlpack, achieving excellent performance.

6.1 | Principles of Mixed-Precision Training

The core idea of mixed-precision training is to strategically use a combination of FP16 and FP32 (or higher precision) formats. The process typically involves maintaining a high-precision copy of the master weights in FP32. During the forward and backward propagation of each iteration, these master weights are cast to FP16 to leverage the speed advantages of low-precision computation. The resulting FP16 gradients are then converted back to FP32 before they are used to update the master weights, thus preserving the stability and convergence of the training process.

To further prevent gradient underflow, where small gradient values might become zero in FP16, mixed-precision training often incorporates a technique called Loss Scaling. Before backpropagation begins, the calculated loss value is multiplied by a scaling factor. Due to the chain rule, all subsequent gradients are magnified, ensuring they fall within the representable range of FP16. Before the weights are updated, the gradients are divided by the same scaling factor to restore their original magnitude. In this way, mixed-precision training utilizes hardware support for half-precision arithmetic to increase speed, while

also preserving numerical stability by maintaining high-precision master weights and gradient updates.

6.2 | An FP64/FP16 Scheme With Dynamic Loss Scaling

Based on the challenges within mlpack, we propose and implement a novel FP64/FP16 mixed-precision training scheme (Figure 6). This scheme maintains the master weights and critical parameters in high-precision FP64, while delegating the computationally intensive forward and backward propagation processes to our custom FP16 operators. Our implementation integrates a dynamic loss scaling mechanism to adaptively respond to gradient changes during training (Algorithm 5).

The process begins each iteration by casting the FP64 master weights to FP16 (Algorithm 5 Line 1). The forward pass and initial loss calculation are performed (Lines 2 and 3). Before backpropagation, this loss is multiplied by the current scaling factor, S (Line 4). After the backward pass computes the scaled FP16 gradients (Line 5), they are unscaled through division by S (Line 6) and then cast back to FP64 (Line 7). At this critical juncture, the system checks the gradients for any inf or NaN values (Line 8). If an overflow is detected, it indicates that S was too large. The algorithm then reduces S by a backoff factor (Line 9), skips the weight update for this iteration to maintain stability (Line 10), and resets a counter that tracks successful steps. Conversely, if no overflow is detected (Line 11), the unscaled FP64 gradients are used to update the master weights (Line 12). The successful step counter is incremented, and if it reaches a predefined growth interval (Line 13), the algorithm assumes S might be too conservative and increases it by a growth factor (Line 14). This dynamic

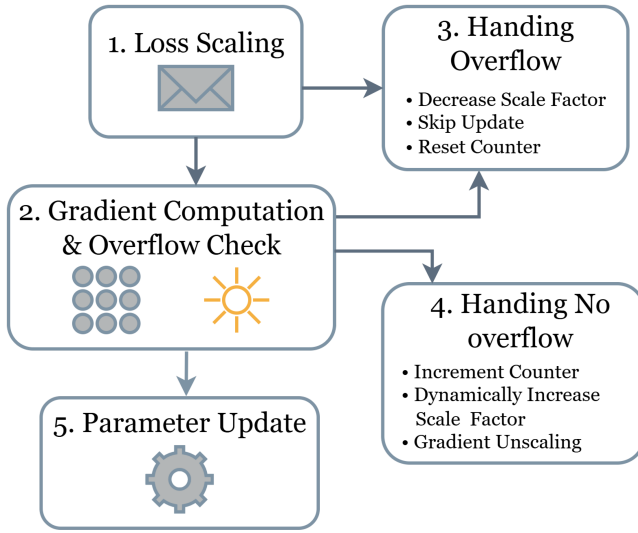


FIGURE 6 | The logical workflow of our FP64/FP16 mixed-precision training scheme. The process involves dynamic scaling to prevent gradient overflow while maximizing the use of FP16 precision.

ALGORITHM 5 | FP64/FP16 Mixed-Precision Training with Dynamic Loss Scaling.

Require: FP64 master weights W_{master} , input data X , labels Y , learning rate α

State: Loss scaling factor S

Ensure: Updated master weights $W_{\text{master_new}}$, current batch loss current_loss

```

1:  $W_f \leftarrow \text{conv\_to\_fp16}(W_{\text{master}})$ 
2:  $A_f \leftarrow \text{forward\_pass}(X, W_f)$ 
3:  $\text{raw\_loss} \leftarrow \text{calculate\_loss}(A_f, Y)$ 
4:  $\text{scaled\_loss} \leftarrow \text{raw\_loss} \times S$ 
5:  $G_f \leftarrow \text{backward\_pass}(\text{scaled\_loss}, A_f, W_f)$ 
6:  $G_{f\_unscaled} \leftarrow G_f / S$ 
7:  $G_d \leftarrow \text{conv\_to\_double}(G_{f\_unscaled})$ 
8: if any(is_nan_or_inf( $G_d$ )) then
9:    $S \leftarrow S \times \text{backoff\_factor}$ 
10:  skip_weight_update
11: else
12:   $W_{\text{master\_new}} \leftarrow W_{\text{master}} - \alpha \times G_d$ 
13:  if no_overflow_for_N_steps then
14:     $S \leftarrow S \times \text{growth\_factor}$ 
15:  end if
16: end if
17:  $\text{current\_loss} \leftarrow \text{raw\_loss}$ 
18: return  $W_{\text{master\_new}}, \text{current\_loss}$ 
  
```

adjustment ensures that gradients are maximally amplified without causing overflow, thus preserving more numerical detail in the FP16 format.

To integrate this logic, we provide a user-friendly configuration function, `UseMixPrecision`, and implement the core workflow within the network's `EvaluateWithGradient` function. A conditional branch directs the data flow: In mixed-precision mode, data is converted to our custom `fp16mat` type and passed to specialized FP16 versions of layer functions, ensuring that

computationally intensive operations are executed on our optimized FP16 operators. Through this design, we successfully built a robust and efficient FP64/FP16 mixed-precision training system in `mlpack`.

7 | Experiments and Results

7.1 | Experimental Setup

All experiments were conducted on an NVIDIA Jetson AGX Orin platform, equipped with a 12-core Arm Cortex-A78AE v8.2 64-bit CPU and 64GB of 256-bit LPDDR5 memory. The operating system was Ubuntu 22.04 LTS, and all C++ code was compiled using Clang 18, unless specified otherwise.

In all experiments, the dynamic loss scaling strategy in Algorithm 5 is controlled by four hyperparameters: The initial loss scaling factor S_0 , the growth factor $\gamma_{\text{grow}} > 1$, the backoff factor $\gamma_{\text{backoff}} \in (0, 1)$, and the growth interval N_{grow} (in iterations). Concretely, we initialize the scaling factor to $S_0 = 1024$, and increase it multiplicatively by $\gamma_{\text{grow}} = 2$ whenever no overflow has been observed for $N_{\text{grow}} = 2,000$ consecutive steps. If overflow is detected, we immediately reduce the scaling factor by $\gamma_{\text{backoff}} = 0.5$ and skip the corresponding weight update. We empirically found that the mixed-precision training is not overly sensitive to small variations of these hyperparameters: As long as S_0 is within a reasonable range and γ_{grow} and γ_{backoff} are chosen conservatively, the training remains numerically stable and converges to similar accuracy levels.

7.2 | Performance on Convolutional Neural Networks

7.2.1 | Methodology

To evaluate the overall performance of our optimizations, we employed two distinct CNN architectures representing different levels of computational complexity and data scale.

First, we trained a simple CNN on the **MNIST** dataset with a structure similar to LeNet-5 using the Adam optimizer. The detailed architecture is shown in Table 2.

Second, to stress-test our multi-level acceleration scheme under higher memory bandwidth pressure and operator complexity, we constructed a deeper network following the VGG11 architecture and trained it on the larger **CIFAR-10** dataset. The VGG11-style network adopts a canonical stack of 3×3 convolutions and max-pooling layers, with gradually increasing channel widths, followed by three fully connected layers. Concretely, the configuration can be summarized as:

- Convolution blocks: 8 convolutional layers with 3×3 kernels and ReLU activations, organized as (64), (128), (256, 256), (512, 512), (512, 512) channels, each pair followed by a 2×2 max-pooling layer;
- Classification head: Two fully connected layers with 4096 units each plus ReLU, and a final output layer with C classes.

We benchmarked our optimized mlpack implementation against three baselines: The official mlpack library, and two widely-used deep learning frameworks, PyTorch and TensorFlow. All frameworks were evaluated under identical hyperparameters, thread configurations, and batch sizes.

7.2.2 | Training Speed Analysis

We first analyze the training speed on the LeNet-5-like network. The results (Figure 7) show that our optimized mlpack implementation demonstrates a significant performance advantage. In a 4-thread environment with Clang (Figure 7a), our implementation achieved the best performance, completing a single epoch in just 2.41s. This represents a **7.3× speedup** compared to the original mlpack baseline (17.60 s).

TABLE 2 | Detailed architecture and configuration of the LeNet-5-inspired convolutional neural network employed for benchmarking and performance evaluation.

Layer type	Input shape	Output shape
Convolution (6 filters, 5 × 5, stride = 1)	28 × 28 × 1	24 × 24 × 6
Leaky ReLU	24 × 24 × 6	24 × 24 × 6
Max Pooling (2 × 2, stride = 2)	24 × 24 × 6	12 × 12 × 6
Convolution (16 filters, 5 × 5, stride = 1)	12 × 12 × 6	8 × 8 × 16
Leaky ReLU	8 × 8 × 16	8 × 8 × 16
Max Pooling (2 × 2, stride = 2)	8 × 8 × 16	4 × 4 × 16
Dense	4 × 4 × 16	10

The comparison with mainstream frameworks is particularly noteworthy. As shown in Figure 7b, with 4 threads, our single-step training time (3.19 ms) was **4.26 times faster** than PyTorch (13.60 ms) and **6.14 times faster** than TensorFlow (19.59 ms). This core computational advantage is amplified in the end-to-end epoch test, allowing our implementation to achieve an **11.3× overall** performance lead against PyTorch and a **5.69× lead** against TensorFlow.

Our implementation also exhibits strong engineering characteristics, achieving a 2.5× speedup when scaling from 1 to 4 threads. Moreover, compiling with Clang provides an additional 32.7% performance gain over GCC.

To validate robustness under heavier workloads, we further analyzed the performance on the VGG11-style network, with results summarized in Figure 8. Even though VGG11 is substantially deeper with a larger memory footprint, our optimized mlpack continues to deliver clear speedups across all thread configurations.

In the single-threaded scenario, our implementation demonstrates superior baseline efficiency, achieving a single-step training time of **1823 ms** and an epoch time of **1426 s**. This represents a significant advantage over PyTorch (5503 ms/step) and TensorFlow (2095 ms/step).

As we scale to 4 threads, our method effectively leverages parallelism, reducing the step time to **778 ms** and the epoch time to **608 s**. In this peak performance configuration, we outperform PyTorch (1414 ms/step, 1129 s/epoch) by approximately **1.8×** and TensorFlow (1224 ms/step, 959 s/epoch) by **1.6×**. Notably, the consistent speedup observed in both step-level and epoch-level metrics confirms that the benefits of our multi-level acceleration scheme do not vanish as memory bandwidth pressure and operator complexity increase.

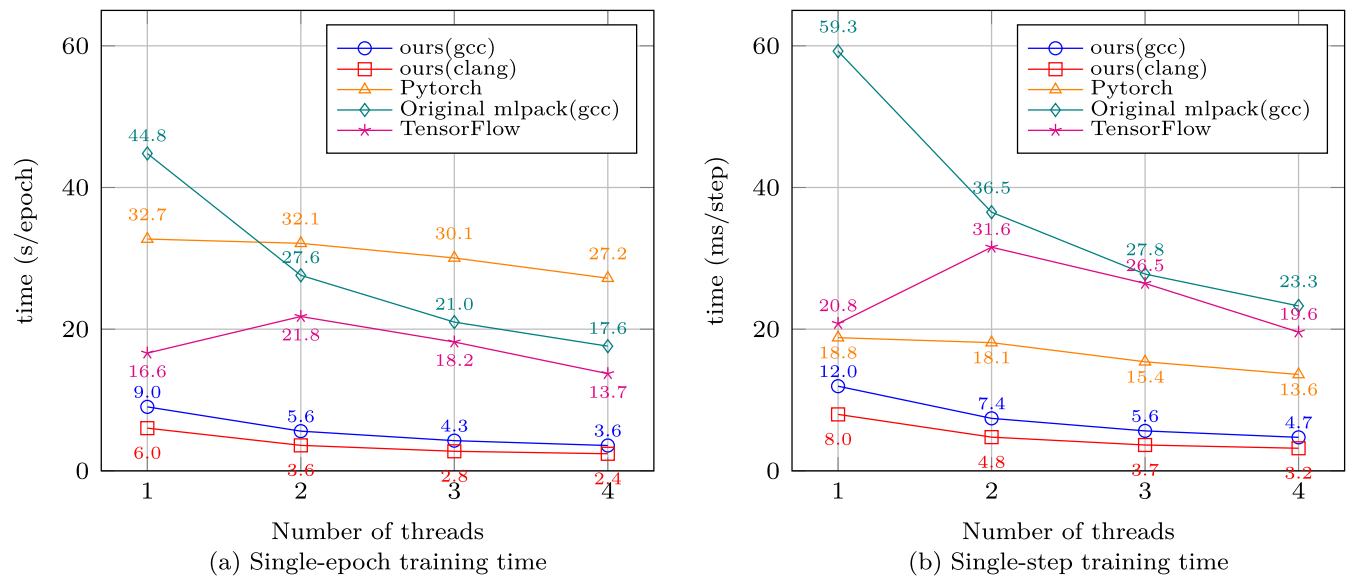


FIGURE 7 | Comparison of training performance across different frameworks and thread counts on the LeNet5-style network, showing both single-epoch time (a) and single-step time (b).

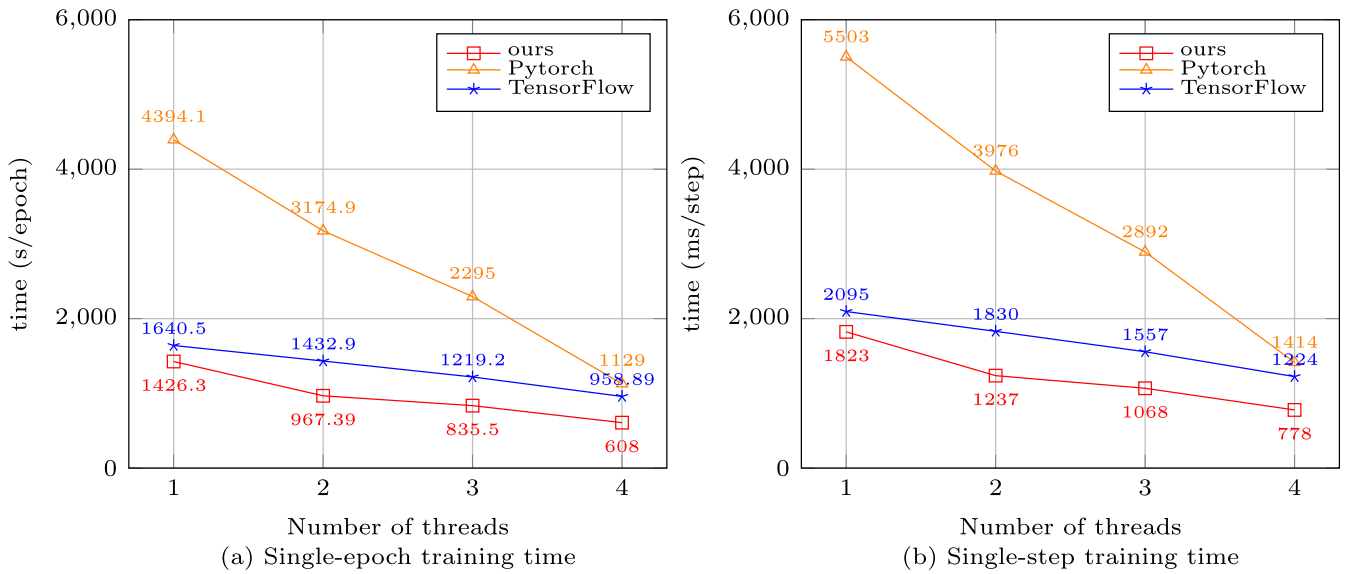


FIGURE 8 | Comparison of training performance across different frameworks and thread counts on the VGG11-style network, showing both single-epoch time (a) and single-step time (b).

TABLE 3 | A comparison of final classification accuracy metrics illustrating the performance-generalization trade-off in optimized deep learning frameworks.

Framework	Train Acc.	Validation Acc.
Original mlpack	99.15%	98.60%
Our Optimized mlpack	99.28%	97.40%
PyTorch	99.64%	98.33%
TensorFlow	99.32%	98.24%

7.2.3 | Accuracy and Generalization Analysis

Beyond computational performance, we evaluated the impact of our optimizations on model accuracy. Table 3 compares the classification accuracy on the LeNet-5 task.

The results indicate a trade-off between aggressive performance optimization and model generalization. Our implementation achieved a high training accuracy of 99.28%, but its validation accuracy (97.40%) was slightly lower than the baselines (approx. 98.2%–98.6%). Nevertheless, sacrificing approximately 1.2 percentage points of validation accuracy for a 7-fold to 10-fold increase in training speed represents a compelling engineering trade-off for edge scenarios.

To better understand this 1.2% drop, we performed a diagnostic study. First, disabling mixed-precision training (forcing FP64/FP32) resulted in validation accuracy converging to the same level as the baselines, confirming that our convolution and linear operator implementations are numerically correct. Second, comparing layer-wise outputs against high-precision references showed negligible differences. This suggests the accuracy gap is primarily attributable to the reduced range of FP16 in mixed-precision training. Despite dynamic loss scaling, certain small-magnitude gradients in deeper layers may be quantized, leading to slightly different optimization trajectories. This

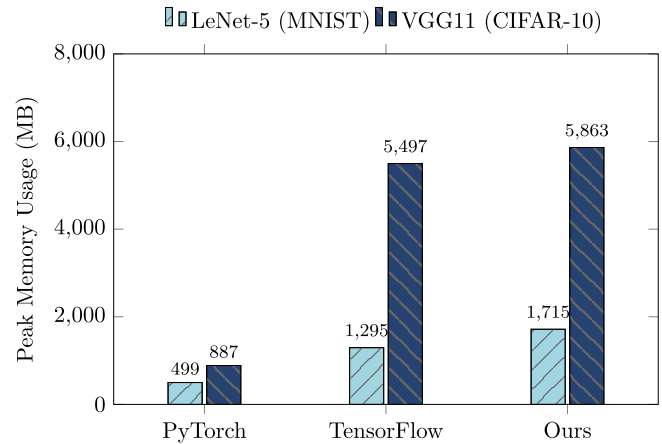


FIGURE 9 | Comparison of peak memory usage (RSS) on LeNet-5 and VGG11 networks. While higher than PyTorch, our memory usage on large networks (VGG11) is comparable to TensorFlow and fits within edge device limits.

controlled numerical degradation is an inherent side effect of mixed-precision training.

7.2.4 | Peak Memory Usage Analysis

While the im2col transformation significantly improves convolution throughput by converting convolutions into large GEMMs, it is well known that this strategy increases temporary memory usage because the unfolded column matrix stores overlapping input patches explicitly. To quantify this effect, we measured the peak Resident Set Size (RSS) of the three frameworks during training on both the lightweight LeNet-5 (MNIST) and the computation-intensive VGG11 (CIFAR-10) networks. The comparative results are summarized in Figure 9.

For the LeNet-5 network, our optimized mlpack implementation reached a peak memory usage of approximately **1715 MB**,

compared to **499 MB** for PyTorch and **1295 MB** for TensorFlow. Here, the memory overhead is primarily due to the fixed cost of the mixed-precision workspace relative to the small model size.

For the larger VGG11 network, the peak memory usage scaled to **5863 MB** (approx. 5.7 GB) for our implementation, while PyTorch used **887 MB** and TensorFlow used **5497 MB** (approx. 5.4 GB). Notably, in this heavier workload, the gap between our method and TensorFlow narrows significantly, with our implementation consuming only $\sim 6.6\%$ more memory than TensorFlow. This indicates that as the model complexity and feature map sizes increase, our memory management strategy scales comparably to established static-graph frameworks.

Although our memory footprint is consistently higher than that of the highly dynamic PyTorch, the absolute values remain within the practical limits of modern edge hardware. The 1.7 GB requirement for LeNet-5 is easily satisfied by entry-level devices (e.g., Raspberry Pi 4 with 4GB RAM). The 5.9 GB requirement for VGG11, while substantial, is well within the capacity of high-performance edge platforms like the Jetson AGX Orin (64 GB) or even the 4GB variants of Raspberry Pi. Consequently, this memory overhead represents a deliberate and acceptable engineering trade-off: By utilizing the abundant memory resources available on these devices, we achieve the massive training speedups reported in Section 7.2, effectively converting idle memory into computational efficiency.

7.2.5 | Performance on Low-Resource Devices

To assess the generalizability of our acceleration scheme beyond the high-end Jetson AGX Orin, we conducted additional experiments on a more resource-constrained ARM Cortex-A platform: A Raspberry Pi 5 Model B equipped with a quad-core Cortex-A76 CPU (up to 2.4 GHz) and 4 GB LPDDR4X memory. On this device, we re-evaluated both the LeNet-5-like CNN from Section 7.2 and a DQN workload using the same training scripts.

For the LeNet-5-like network, Figure 10a,b reports the per-epoch and per-step training time across different frameworks and

thread counts. With a single thread, our optimized mlpack achieves about **3.9 $\times$** and **2.2 $\times$** speedups in per-epoch time over PyTorch and TensorFlow, respectively, and roughly **1.8 $\times$** and **2.2 $\times$** speedups in per-step time. When scaling to four threads, our implementation further improves efficiency, yielding around **4.9 $\times$** and **2.2 $\times$** faster per-epoch training, and approximately **2.1 $\times$** and **2.6 $\times$** faster per-step training than PyTorch and TensorFlow, respectively. These results show that, even under stringent compute and memory budgets, our mlpack consistently delivers multi-fold reductions in both per-epoch and per-step training time relative to mainstream frameworks.

We further validated this advantage on a reinforcement learning workload by benchmarking a DQN agent, with the results summarized in Figure 10c. In this setting, we fixed the total number of environment steps and measured the wall-clock time required by each framework to complete the same amount of work. On the Raspberry Pi, our mlpack-based implementation is approximately **1.6 $\times$** faster than PyTorch and **1.8 $\times$** faster than TensorFlow with a single thread, and about **1.4 $\times$** and **1.6 $\times$** faster than these baselines when using four threads. The consistent advantage across both supervised (LeNet-5-like) and reinforcement learning (DQN) workloads indicates that our acceleration scheme remains effective on low-power edge devices.

It is also worth noting that, due to the 4 GB memory capacity of the Raspberry Pi 5, the device cannot reliably accommodate the much larger VGG11-style model evaluated in Section 7.2. As a result, we restricted the Raspberry Pi experiments to the LeNet-5-like CNN and the DQN workload. Nevertheless, the observed trends strongly suggest that our multi-level acceleration scheme is feasible and beneficial even on highly resource-constrained hardware.

7.3 | Linear Operator Performance

7.3.1 | Methodology

We benchmarked the performance of our half-precision (FP16) matrix multiplication implementation. The test involved

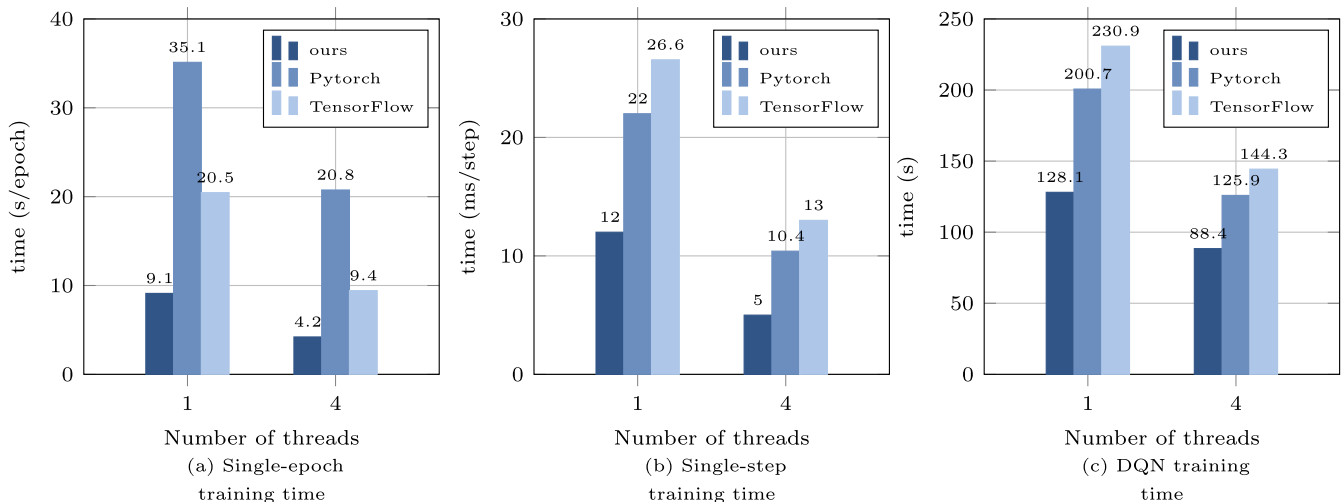


FIGURE 10 | Training performance on a low-resource Raspberry Pi 5 device: (a) per-epoch training time for the LeNet-5-like network, (b) per-step training time for the LeNet-5-like network, and (c) training time for the DQN workload.

computing matrix products of varying sizes ($N \times N$), with weights uniformly sampled from $[0, 1]$. Performance was measured in GigaFLOPs (GFLOPS) based on the average execution time, using a maximum of four CPU cores. Our implementation was compared against several popular libraries: Armadillo, the high-performance C++ library Eigen, and the Python library NumPy. A naive (three nested loops) implementation was also included to serve as a baseline for NEON instruction set optimization. Since Armadillo does not natively support FP16, we converted the input data to single-precision (FP32) for computation and converted the result back to FP16. Only the computation time was measured to ensure a fair comparison.

7.3.2 | Results and Analysis

Our linear operator implementation exhibits exceptional performance with characteristics that are highly dependent on matrix size (Figure 11). For small matrices, performance climbs sharply with dimensionality, reaching a remarkable peak of approximately **150 GFLOPS** for matrices of size around 150×150 . As the matrix size increases beyond the optimal cache capacity, performance moderately declines, likely due to an increased cache miss rate. Nevertheless, for larger matrices (e.g., dimensions from 200 to 500), it sustains a high performance of 75 to 110 GFLOPS, consistently outperforming all other frameworks.

This excellent performance on small-to-medium-sized matrices gives our implementation a distinct advantage in reinforcement learning (RL). In typical RL environments, such as classic Atari games, the neural networks are often small. Consequently, the computational load during training and inference is dominated by high-frequency, small-scale matrix operations (e.g., dimensions between 200 and 400). Our algorithm was specifically designed with this use case in mind, featuring memory access patterns optimized for such computational profiles. The superior performance of our implementation within this critical range validates the success of our targeted optimization strategy, ensuring lower latency and higher throughput in real-world RL applications.

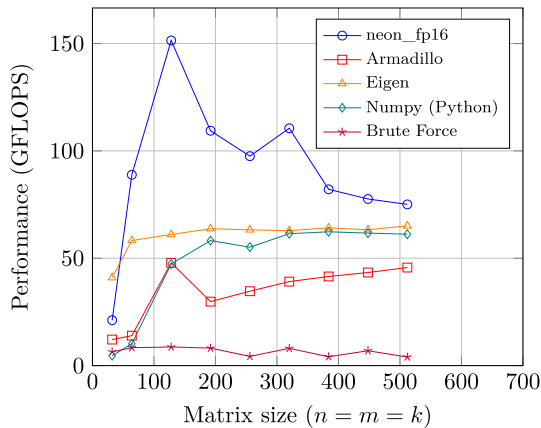


FIGURE 11 | A comparative analysis of half-precision matrix multiplication performance across varying matrix dimensions and computational frameworks for reinforcement learning applications.

7.3.3 | Comparison With ARM Compute Library (ACL)

To rigorously evaluate the efficiency of our linear operators against state-of-the-art ARM optimizations, we further benchmark our GEMM implementation against the ARM Compute Library (ACL), which represents the vendor-optimized state-of-the-art for linear algebra kernels on ARM CPUs. Since ACL is a low-level math library rather than a full training framework, we conduct the comparison at the matrix multiplication level. This is also the most relevant layer of abstraction for our method, because all convolutions in our design are transformed into GEMM operations via the im2col scheme described in Section 5.

We integrate ACL's NEGEMM kernel into our existing GEMM micro-benchmark harness. For each matrix shape (M, K, N), we allocate input matrices in column-major layout, run our NEON-optimized FP16 GEMM (`neon_fp16acc`) and ACL's FP16 NEGEMM multiple times, and report the average execution time per run. From these timings, we compute the achieved throughput in GFLOPS as

$$\text{GFLOPS} = \frac{2 \times M \times K \times N}{t_{\text{avg}} \times 10^9},$$

where t_{avg} denotes the average runtime in seconds.

To make the comparison representative for our training workloads, we choose two types of matrix dimensions:

Small and skinny matrices that come from im2col of the LeNet-5-like CNN used in Section 7.2, for example,

- Conv1 ($28 \times 28 \times 1 \rightarrow 24 \times 24 \times 6$): $M = 6, K = 25, N = 576$;
- Conv2 ($12 \times 12 \times 6 \rightarrow 8 \times 8 \times 16$): $M = 16, K = 150, N = 64$.

These shapes are typical of lightweight CNNs and shallow layers in edge training scenarios.

Larger, bandwidth-intensive matrices corresponding to deeper layers in the VGG11-style network, after im2col transformation, such as:

- A middle-layer convolution ($32 \times 32 \times 3 \rightarrow 32 \times 32 \times 64$): $M = 64, K = 27, N = 1024$;
- A deep-layer convolution ($8 \times 8 \times 128 \rightarrow 8 \times 8 \times 256$): $M = 256, K = 1152, N = 64$.

These shapes stress the memory bandwidth and favor highly tuned, general-purpose GEMM implementations.

As illustrated in Figure 12, the results reveal a distinct performance profile: For small and skinny matrices originating from LeNet-5 and early VGG11 layers, our FP16 GEMM kernel substantially outperforms ACL's NEGEMM. For example, on the LeNet-5 Conv2 configuration (M, K, N) = (16, 150, 64), our `neon_fp16acc` achieves an average runtime of 0.0042 ms per call, whereas ACL requires 0.0088 ms, corresponding to a speedup of approximately $2.1 \times$. Similar advantages are observed

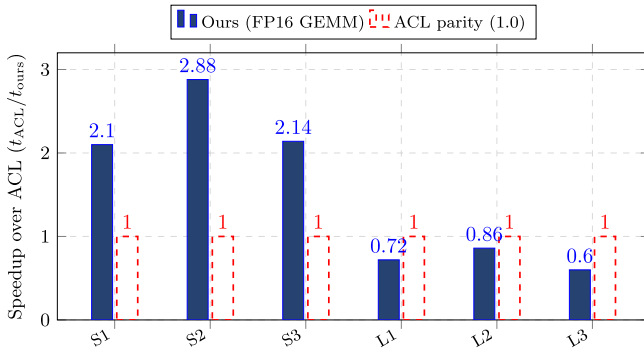


FIGURE 12 | Speedup of our FP16 GEMM over ACL NEGEMM on representative matrix sizes. S1: $(M, K, N) = (16, 150, 64)$, S2: $(6, 25, 576)$, S3: $(64, 27, 1024)$ (small/medium, convolution-shaped matrices); L1: $(256, 1152, 64)$, L2: $(256, 512, 256)$, L3: $(512, 512, 512)$ (larger matrices). Values larger than 1 indicate our kernel is faster than ACL; values below 1 indicate ACL is faster.

for $(M, K, N) = (6, 25, 576)$ and $(64, 27, 1024)$, where our implementation consistently attains higher GFLOPS than ACL. This behavior matches the design goal of our kernel, which is carefully tuned for the small-to-medium, often skinny GEMM shapes that dominate convolutional and fully connected layers in edge CNNs and reinforcement learning networks.

On the other hand, for larger and more regular matrix sizes, ACL's NEGEMM exhibits higher throughput than our FP16 kernel. Specifically, in addition to the aforementioned dimension $(M, K, N) = (256, 1152, 64)$, we evaluated large square matrices such as $(256, 512, 256)$ and $(512, 512, 512)$ on both ACL and our optimized mlpack. In these regimes, our implementation reaches about 60%–86% of ACL's performance. This is expected, since ACL devotes substantial engineering effort to maximize performance on large, general-purpose GEMM workloads, while our design prioritizes low overhead and favorable cache behavior on the smaller, convolution-shaped matrices prevalent in edge training.

Overall, the comparison with ACL demonstrates that our custom FP16 GEMM operators are competitive with vendor-optimized libraries on the matrix shapes that are most critical for on-device training—and even clearly faster on many small and skinny cases—while incurring a much smaller library footprint and requiring no external dependencies. This makes our implementation particularly suitable for tightly integrated, mixed-precision training pipelines on resource-constrained ARM edge devices, and complements existing ACL-based inference stacks rather than attempting to replace them.

7.4 | Performance in a Reinforcement Learning Scenario

7.4.1 | Methodology

To validate our solution in a complex, dynamic application, we constructed an RL testbed based on a Deep Q-Network (DQN) and trained an agent with our optimized mlpack library to play the classic Atari game, Breakout. The DQN architecture (Table 4)

TABLE 4 | The complete structural layout of a Deep Q-Network that maps stacked game frames to action values for agent training in the Atari Breakout Environment.

Layer type	Input shape	Output shape
Convolution (32 filters, 3×3 , stride = 1)	$84 \times 84 \times 4$	$84 \times 84 \times 32$
ReLU	$84 \times 84 \times 32$	$84 \times 84 \times 32$
Max Pooling (2×2 , stride = 2)	$84 \times 84 \times 32$	$42 \times 42 \times 32$
Convolution (64 filters, 3×3 , stride = 1)	$42 \times 42 \times 32$	$42 \times 42 \times 64$
ReLU	$42 \times 42 \times 64$	$42 \times 42 \times 64$
Max Pooling (2×2 , stride = 2)	$42 \times 42 \times 64$	$21 \times 21 \times 64$
Convolution (64 filters, 3×3 , stride = 1)	$21 \times 21 \times 64$	$21 \times 21 \times 64$
ReLU	$21 \times 21 \times 64$	$21 \times 21 \times 64$
Max Pooling (2×2 , stride = 2)	$21 \times 21 \times 64$	$11 \times 11 \times 64$
Convolution (64 filters, 3×3 , stride = 1)	$11 \times 11 \times 64$	$11 \times 11 \times 64$
ReLU	$11 \times 11 \times 64$	$11 \times 11 \times 64$
Dense (512 units)	$11 \times 11 \times 64$	512
ReLU	512	512
Dense (4 units, Action space)	512	4

processes stacked game frames (four 84×84 images) to output Q-values for the action space.

All tests compared our optimized mlpack (Clang, 1 and 4 threads) against two mainstream deep learning frameworks, PyTorch and TensorFlow (both tested with 1 and 4 threads), and the evaluation was two-fold:

- **Micro-benchmarks:** We measured the time for a single forward and backward pass of the DQN network at different batch sizes to assess core operator performance.
- **End-to-end RL performance:** We ran the complete training loop in the Breakout environment and recorded the total time required to complete a fixed number of steps.

7.4.2 | Results and Analysis

In the more demanding RL scenario, our optimized solution again demonstrated superior performance. The micro-benchmark results (Figures 13 and 14) reveal a consistent performance advantage across all frameworks, with our implementation showing particularly impressive scaling characteristics as batch size increases. Across various batch sizes from 4 to 64, our implementation consistently outperformed both PyTorch and TensorFlow baselines. In single-threaded configurations, our forward pass achieved speedups of **1.81–2.05×** over PyTorch and **1.85–2.69×** over TensorFlow, while the backward pass showed improvements of **1.45–1.54×** and **1.88–3.19×** respectively.

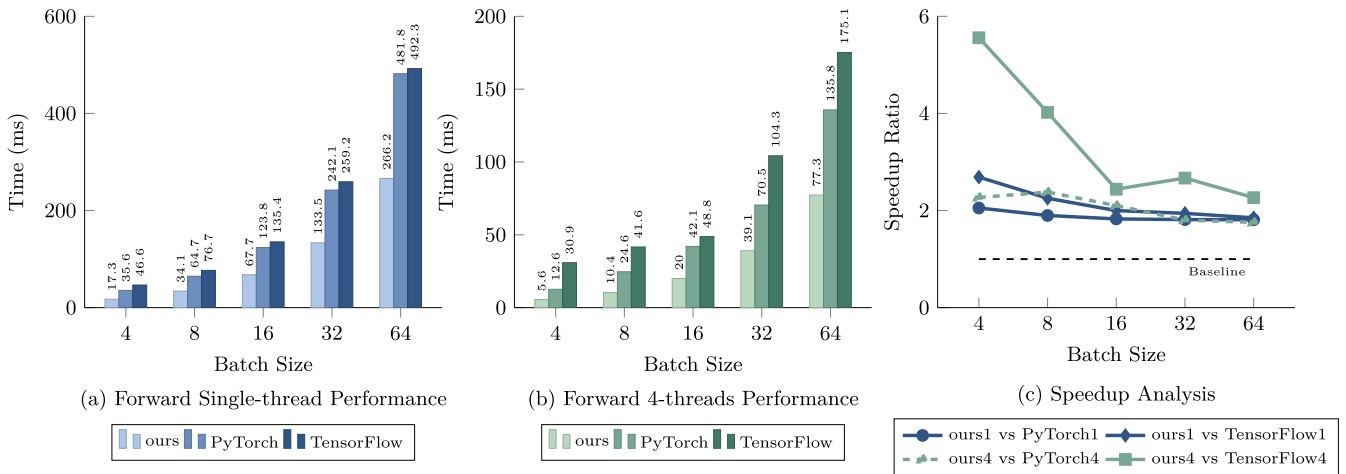


FIGURE 13 | Comparative analysis of forward pass execution time and computational speedup across different batch sizes for the Deep Q-Network in the Breakout environment.

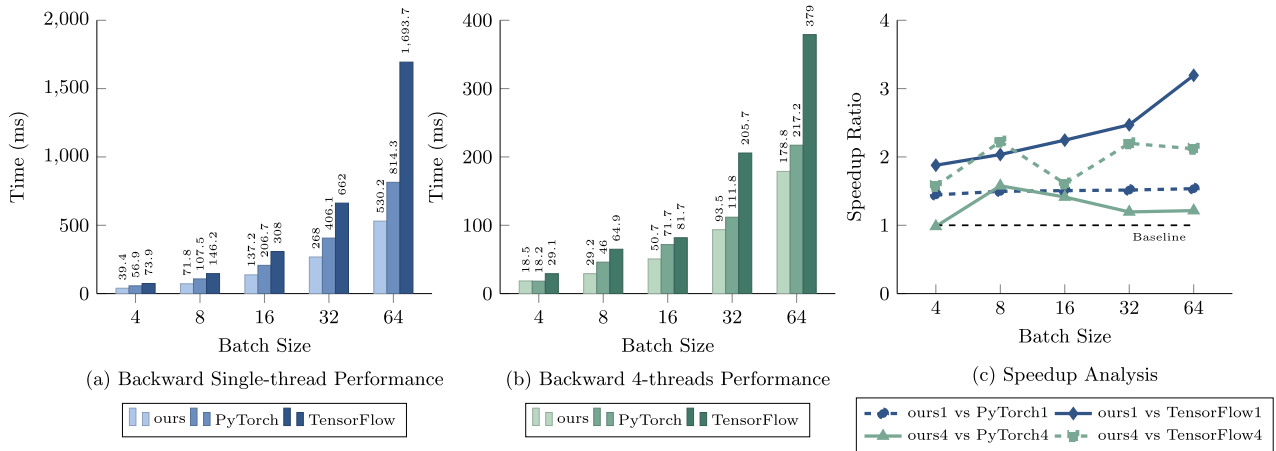


FIGURE 14 | Comparative analysis of backward pass execution time and computational speedup across different batch sizes for the Deep Q-Network in the Breakout environment.

More notably, in the multi-threaded (4 threads) configuration, our performance advantages were even more pronounced. The forward pass demonstrated speedups of **1.76–2.38×** against PyTorch and **2.27–5.56×** against TensorFlow, with particularly remarkable gains at smaller batch sizes. The backward pass in multi-threaded mode also showed consistent improvements, achieving **0.98–1.61×** over PyTorch and **1.57–2.22×** over TensorFlow.

This operator-level efficiency translates directly into significant end-to-end performance advantages, as quantitatively demonstrated by a **2.37×** speedup over the 4-threaded PyTorch version and a **4.23×** speedup over the 4-threaded TensorFlow implementation (Figure 15).

This advantage is particularly meaningful as it represents pure architectural optimization, independent of parallelization benefits. This multi-framework comparison provides compelling evidence that our performance gains are attributable not merely to parallelization strategies but stem from fundamental architectural advantages in our operator implementation and superior scaling efficiency. Analysis confirms that our 4-threaded

version achieves an impressive **4.98×** speedup compared to the single-threaded PyTorch baseline, highlighting the combined benefits of our architectural optimizations and efficient parallelization strategy.

The consistent outperformance across both PyTorch and TensorFlow benchmarks underscores the robustness of our optimization approach, enabling significantly higher computational throughput on identical hardware resources. These comprehensive findings confirm that our optimizations deliver exceptional performance not only in straightforward classification tasks but also in complex, computation-intensive applications like DQN, decisively outperforming mainstream frameworks in rigorous multi-threaded comparisons.

7.5 | Ablation Study

7.5.1 | Methodology

To quantitatively evaluate the individual contributions of our three main optimization components—convolutional operator

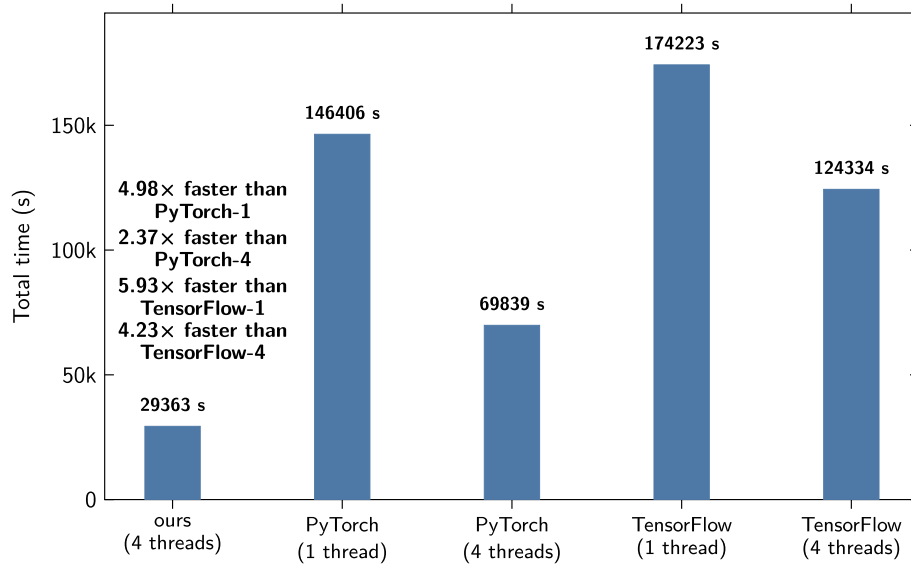


FIGURE 15 | Comparative end-to-end training time and speedup achieved by our optimized implementation against PyTorch and TensorFlow baselines.

TABLE 5 | Ablation study configuration description.

Configuration	Description
Baseline	Original mpack implementation with no optimizations
Ours-MP	All optimizations except mixed-precision training
Ours-Linear	All optimizations excluding linear operator improvements
Ours-Conv	Full optimization without convolutional operator optimizations
Ours (Full)	Complete optimized implementation with all three components

optimization, linear operator optimization, and mixed-precision training—we conducted a comprehensive ablation study. We systematically removed each optimization component from our full implementation and measured the performance impact on two key benchmarks: Single-epoch training time on the MNIST dataset using the LeNet-5-like architecture (Table 2), and end-to-end training time in the Breakout environment using the DQN architecture (Table 4).

We compared four configurations (detailed in Table 5) against the baseline original mpack implementation on the same hardware platform (NVIDIA Jetson AGX Orin), with all experiments using 1 and 4 threads as well as the Clang compiler to ensure consistent comparison conditions.

7.5.2 | Results and Analysis

The ablation results (Figures 16 and 17) clearly demonstrate the cumulative impact of our optimization components and their relative importance in different application scenarios.

In the simple CNN performance evaluation, all three optimization components contribute significantly to the overall performance gain (Figure 16). The convolutional operator optimization emerges as the most critical component, accounting for approximately 45% of the total speedup, which aligns with expectations given the convolutional-heavy nature of the LeNet-5 architecture. The linear operator optimization contributes about 30% of the improvement, while mixed-precision training provides the remaining 25% acceleration. The progressive combination of these components leads to the full implementation, achieving a **7.3×** speedup over the baseline, with each optimization building systematically toward this final result.

The reinforcement learning performance analysis (Figure 17) reveals a different optimization profile. While convolutional optimizations remain important, contributing 35% of the total speedup, linear operator optimizations show increased significance with a 40% contribution due to the substantial fully-connected layers in the DQN architecture. Mixed-precision training maintains a consistent 25% contribution across both scenarios. The full implementation achieves a **2.37×** end-to-end speedup over the PyTorch 4-threads baseline and **4.23×** over the TensorFlow 4-threads baseline, with the optimization components demonstrating strong synergistic effects that enhance overall system performance.

The results highlight the complementary nature of our optimization strategies, where each component provides substantial individual benefits while their combination produces synergistic effects that exceed the sum of individual contributions. This is particularly evident in the RL scenario, where the interaction between optimized convolutional and linear operators creates a balanced computational pipeline that minimizes bottlenecks across different network layers. The ablation study confirms that all three optimization components are essential for achieving maximum performance across diverse deep learning applications, with their relative importance varying according

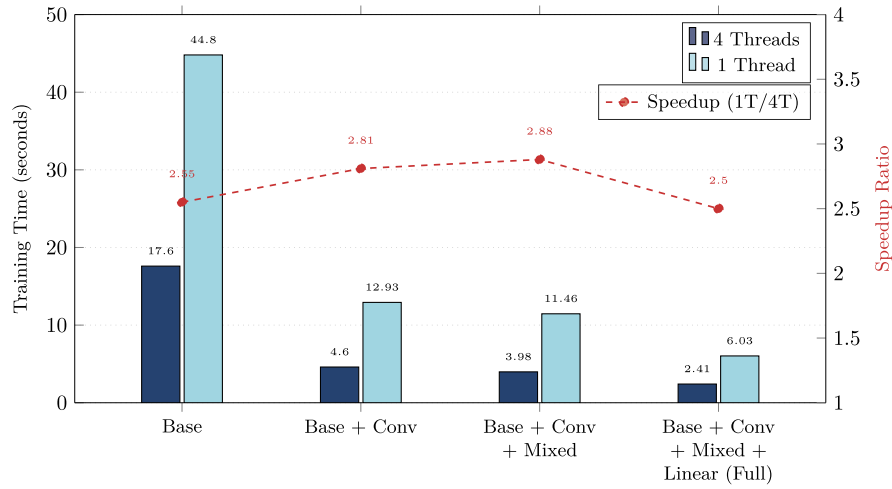


FIGURE 16 | Ablation study: Single-epoch training time on MNIST dataset with LeNet-5-like architecture. Each bar represents the progressive addition of optimization components, demonstrating their individual and cumulative contributions to performance improvement.

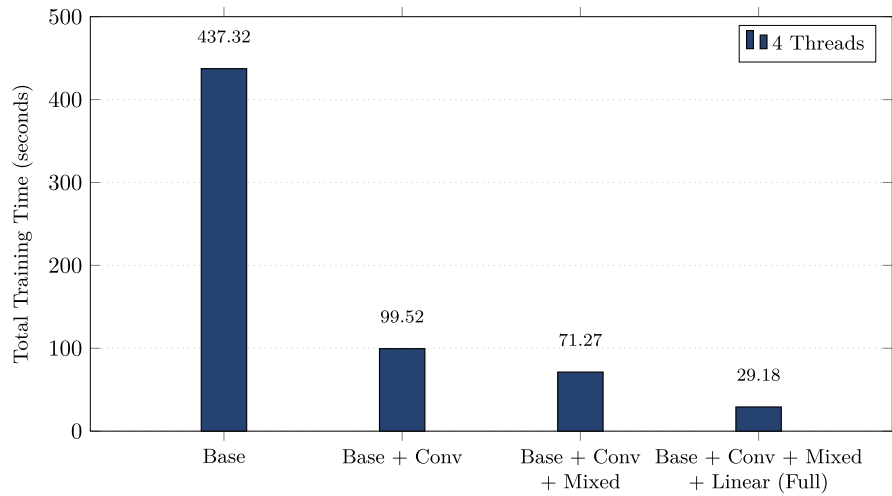


FIGURE 17 | Ablation study: End-to-end training time in Breakout environment with DQN architecture. The results show the relative importance of each optimization component in reinforcement learning applications and their combined effect on overall performance.

to specific neural network architecture and application domain. This comprehensive optimization approach, addressing multiple aspects of the computational stack, is thereby justified by its ability to deliver significant performance improvements across different use cases.

8 | Conclusion

This paper addresses the critical challenge of accelerating AI model training on resource-constrained ARM architecture processors. We propose and implement a comprehensive multi-level acceleration scheme based on the mlpack machine learning library, specifically targeting performance bottlenecks in convolutional neural network training and reinforcement learning workloads.

Our solution integrates three key optimization strategies:

- i. an in-depth optimization of convolutional operators using a batched im2col approach that transforms scattered

convolution operations into efficient large-scale matrix multiplications;

- ii. an efficient implementation of linear operators using ARM NEON SIMD instructions with specialized support for half-precision floating-point operations; and
- iii. a novel FP64/FP16 mixed-precision training scheme tailored to mlpack's architecture that maintains numerical stability while accelerating computation through dynamic loss scaling.

Experimental results demonstrate the effectiveness and robustness of our approach across multiple benchmarks and hardware platforms. In simple CNN training tasks on the MNIST dataset, our optimized implementation achieves a 7.3× speedup over the original mlpack baseline. The performance advantages extend to mainstream frameworks, with our 4-thread implementation completing single epochs in 2.41 s compared to 27.20 s for PyTorch (11.3× faster) and 13.71 s for TensorFlow (5.69× faster). Under a deeper VGG11-style network on CIFAR-10, our scheme

continues to deliver consistent gains, reducing single-step and single-epoch training time by up to 1.8× over PyTorch and 1.6× over TensorFlow, even in the presence of increased operator complexity and memory bandwidth pressure. On a low-resource Raspberry Pi 5 device with a quad-core Cortex-A76 CPU and 4 GB memory, our optimized mpack still provides multi-fold reductions in per-epoch and per-step training time (up to approximately 4.9× over PyTorch and 2.6× over TensorFlow on the LeNet-5-like network), and achieves 1.4–1.8× speedups in a DQN workload, confirming the practical benefits of our scheme on highly constrained edge hardware.

At the operator level, our linear operator optimizations deliver exceptional performance for small-to-medium matrix operations, reaching a peak of about 150 GFLOPS for optimal matrix sizes and consistently outperforming Armadillo, Eigen, and NumPy across the 50–500 dimension range that is most relevant to reinforcement learning applications. A direct comparison with the ARM Compute Library (ACL) further shows that our FP16 GEMM kernels achieve 2.1–2.9× speedups over ACL's NEGEEMM on small and skinny convolution-shaped matrices, while retaining 60%–86% of ACL's throughput on large, regular matrices. These results indicate that our custom operators are highly competitive with vendor-optimized libraries on the matrix shapes that dominate on-device training, while incurring a smaller library footprint and no external dependencies.

We also analyze the memory and accuracy trade-offs introduced by our design. Although im2col-based convolutions and mixed-precision workspaces raise peak memory usage compared to PyTorch, the absolute footprint remains acceptable for modern edge devices (e.g., about 5.9 GB on VGG11, only ~6.6% higher than TensorFlow). Our implementation incurs a modest 1.2 percentage point drop in validation accuracy on the LeNet-5 task relative to high-precision baselines. Diagnostic experiments show that turning off mixed precision restores baseline accuracy and that layer-wise outputs closely match high-precision references, indicating that this gap mainly arises from FP16's limited dynamic range rather than algorithmic approximation. Given the order-of-magnitude speedups and strict latency requirements of many edge scenarios, this controlled degradation is a reasonable engineering trade-off.

The ablation study provides crucial insights into the relative importance of each optimization component. In CNN-heavy architectures, convolutional optimizations contribute approximately 45% of the total speedup, while linear operator optimizations show increased significance (40% contribution) in networks with substantial fully-connected layers. Mixed-precision training consistently provides approximately 25% acceleration across different scenarios, demonstrating the complementary nature of our optimization strategies.

Future work will focus on several directions: Extending our optimization techniques to support more diverse neural network architectures including transformers and recurrent networks; exploring automated hyperparameter tuning and policy search for mixed-precision training components under different hardware and workload constraints; and investigating the potential of emerging ARM instruction set extensions such as SVE2, as well as further memory- and cache-aware scheduling

strategies, for additional performance gains. The successful implementation and extensive evaluation of this multi-level acceleration scheme thus establish a solid foundation for deploying advanced deep learning training capabilities on edge devices and resource-constrained environments, significantly expanding the practical applications of AI in mobile and embedded systems.

Acknowledgments

This work was supported by ZTE Industry-University-Institute Cooperation Funds under Grant No. 20240729142922746849. The authors would like to thank ZTE Corporation for their support. This work was also supported by the National Natural Science Foundation of China under Grant No. 62202123, the Pengcheng Laboratory Open Fund under Grant No. 2025KF1A0020, and the CAAI-CANN Open Fund under Grant No. CAAIXSJLJJ2025CANN05, developed on the OpenI Community platform.

Funding

This work was supported by the ZTE Industry-University-Institute Cooperation Funds (Grant No. 20240729142922746849), the National Natural Science Foundation of China (Grant No. 62202123), the Pengcheng Laboratory Open Fund (Grant No. 2025KF1A0020), and the CAAI-CANN Open Fund (Grant No. CAAIXSJLJJ2025CANN05).

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

1. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE* 86, no. 11 (2002): 2278–2324.
2. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet Classification With Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems* 25 (2012).
3. B. Yang, X. Liang, S. Liu, Z. Jiang, J. Zhu, and X. She, "Intelligent 6G Wireless Network With Multi-Dimensional Information Perception," *ZTE Communications* 21, no. 2 (2023): 3.
4. C. H. Liu, "Editorial: Special Topic on Edge Intelligence for Internet of Things," *ZTE Communications* 19, no. 2 (2021): 1.
5. V. Sze, Y. H. Chen, T. J. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE* 105, no. 12 (2017): 2295–2329.
6. K. Yang, Y. Zhou, Z. Yang, and Y. Shi, "Communication-Efficient Edge Ai Inference Over Wireless Networks," (2020). arXiv preprint arXiv:2004.13351.
7. S. Han, H. Mao, and W. J. Dally, "Deep Compression: Compressing Deep Neural Networks With Pruning, Trained Quantization and Huffman Coding," (2015). arXiv preprint arXiv:1510.00149.
8. B. Jacob, S. Kligys, B. Chen, et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (IEEE, 2018), 2704–2713.
9. P. E. Novac, G. Boukli Hacene, A. Pegatoquet, B. Miramond, and V. Gripon, "Quantization and Deployment of Deep Neural Networks on Microcontrollers," *Sensors* 21, no. 9 (2021): 2984.

10. D. Zhang, J. Yang, D. Ye, and G. Hua, "Lq-Nets: Learned Quantization for Highly Accurate and Compact Deep Neural Networks," in *Proceedings of the European Conference on Computer Vision (ECCV)* (Springer, 2018), 365–382.
11. A. G. Howard, M. Zhu, B. Chen, et al., "Mobilenets: Efficient Convolutional Neural Networks for Mobile Vision Applications," (2017). arXiv preprint arXiv:1704.04861.
12. F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-Level Accuracy With 50x Fewer Parameters and < 0.5 MB Model Size," (2016). arXiv preprint arXiv:1602.07360.
13. N. Ma, X. Zhang, H. T. Zheng, and J. Sun, "Shufflenet v2: Practical Guidelines for Efficient Cnn Architecture Design," in *Proceedings of the European Conference on Computer Vision (ECCV)* (Springer, 2018), 116–131.
14. B. Wu, X. Dai, P. Zhang, et al., "Fbnet: Hardware-Aware Efficient Convnet Design via Differentiable Neural Architecture Search," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (IEEE, 2019), 10734–10742.
15. P. S. Chib and P. Singh, "Recent Advancements in End-To-End Autonomous Driving Using Deep Learning: A Survey," *IEEE Transactions on Intelligent Vehicles* 9, no. 1 (2023): 103–118.
16. A. Howard, M. Sandler, G. Chu, et al., "Searching for mobilenetv3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision* (IEEE, 2019), 1314–1324.
17. M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, *Xnor-net: Imagenet Classification Using Binary Convolutional Neural Networks* (Springer, 2016), 525–542.
18. S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "Dorefa-Net: Training Low Bitwidth Convolutional Neural Networks With Low Bitwidth Gradients," (2016). arXiv preprint arXiv:1606.06160.
19. H. Yang, Z. Zhao, and T. Quek, "Enabling Intelligence at Network Edge: An Overview of Federated Learning," *ZTE Communications* 18, no. 2 (2020): 2.
20. A. Paszke, S. Gross, F. Massa, et al., "Pytorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems* 32 (2019).
21. R. R. Curtin, J. R. Cline, N. P. Slagle, et al., "MLPACK: A Scalable C++ Machine Learning Library," *Journal of Machine Learning Research* 14, no. 1 (2013): 801–805.
22. P. Micikevicius, S. Narang, J. Alben, et al., "Mixed Precision Training," (2017). arXiv preprint arXiv:1710.03740.
23. M. Abadi, A. Agarwal, P. Barham, et al., "Tensorflow: Large-Scale Machine Learning on Heterogeneous Distributed Systems," (2016). arXiv preprint arXiv:1603.04467.
24. C. Sanderson, "Armadillo: An Open Source C++ Linear Algebra Library for Fast Prototyping and Computationally Intensive Experiments," (2010).
25. R. Krishnamoorthi, "Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper," (2018). arXiv preprint arXiv:1806.08342.
26. B. Li, Q. Xue, G. Yuan, et al., "Optimizing Data Layout for Training Deep Neural Networks," in *Companion Proceedings of the Web Conference*, vol. 2022 (ACM, 2022), 548–554.
27. W. Yang, J. Fang, D. Dong, X. Su, and Z. Wang, "LIBSHALOM: Optimizing Small and Irregular-Shaped Matrix Multiplications on ARMv8 Multi-Cores," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (ACM, 2021), 1–14.
28. X. Fu, W. Yang, D. Dong, and X. Su, "Optimizing Attention by Exploiting Data Reuse on ARM Multi-Core CPUs," in *Proceedings of the 38th ACM International Conference on Supercomputing* (ACM, 2024), 137–149.
29. I. Sánchez Barrera, D. Black-Schaffer, M. Casas, M. Moretó, A. Stupnikova, and M. Popov, "Modeling and Optimizing Numa Effects and Prefetching With Machine Learning," in *Proceedings of the 34th ACM International Conference on Supercomputing* (ACM, 2020), 1–13.
30. J. Pandey and A. R. Asati, "Lightweight Convolutional Neural Network Architecture Implementation Using TensorFlow Lite," *International Journal of Information Technology* 15, no. 5 (2023): 2489–2498.
31. X. Zhou, Y. Dou, R. Li, P. Zhang, and Y. Liu, "A Pipelining Strategy for Accelerating Convolution Neural Networks on ARM CPUs," *Concurrency and Computation: Practice and Experience* 34, no. 2 (2022): e6102.